

Министерство цифрового развития, связи и массовых коммуникаций Российской Федерации
федеральное государственное бюджетное образовательное учреждение высшего образования
«Сибирский государственный университет телекоммуникаций и информатики»
(СибГУТИ)

09.03.01 — Информатика и вычислительная техника
(направление подготовки/специальность)

Программное обеспечение мобильных систем
(профиль/специализация)

Очная
(форма обучения)

ОТЧЕТ ПО ПРОИЗВОДСТВЕННОЙ ПРАКТИКЕ

(вид практики)

Тип практики: Технологическая (проектно-технологическая) практика
на предприятии ООО «Бюро 1440»

(наименование профильной организации/структурного подразделения СибГУТИ)

ТЕМА ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

Пока ничего

Выполнил:

студент института информатики и вычислительной техники Рубцов Роман Кириллович
группа ИА-331

«___» _____ 202__ г.

_____/_____/_____
(подпись) (ФИО)

Проверил¹

Руководитель практики от профильной организации

«___» _____ 202__ г.

_____/ Андреев А. В. /
(подпись) (ФИО)

Проверил:

Руководитель практики от СибГУТИ

«___» _____ 202__ г.

_____/ Брагин К. И. /
(подпись) (ФИО)

отметка² _____

_____ «___» _____ 202__ г.

Новосибирск 2025

¹ В случае прохождения практики в профильной организации

² Заполняется во время промежуточной аттестации

Рубцов Роман Кириллович
Фамилия Имя Отчество студента

Тема индивидуального задания практики: Пока ничего (SDR)

³В случае прохождения практики в профильной организации

AAAAAAAAAAAA AAAAAAAAAAAAAA AAAAAAAAAAAAAA AAAAAAAAAAAAAAAAAAAA AAAA	AA.AA
AAAAAAA AAAAAAAAAAAAAA AAAAAAAAAAAAAAAAAA AAAAAAAAAAAAAAAAAAAA	AA.AA
AAAAAAAAAAAA AAAAAAAAAAAAAA AAAAAAAAAAAAAA AAAAAAAAAAAA AAAAAA	AA.AA
AAAAAA AAAAAAAAAAAAAA AAAAAAAAAAAAAAAAAA AAAAAAAAAAAAAAAAAAAA AAAAA	AA.AA

В соответствии с рабочей программой практики
Руководитель практики от профильной организации*

«___» _____ 202__ г.

_____/ Андреев А. В. /
(подпись) (ФИО)

Руководитель практики от СибГУТИ

«___» _____ 202__ г.

_____/ Брагин К. И. /
(подпись) (ФИО)

Отзыв о работе студента

(ФИО студента)

Уровень освоения компетенций

Компетенции	Уровень сформированности компетенций
ПК-1 Способен разрабатывать требования и проектировать программное обеспечение	
ПК-3 Способен осуществлять эксплуатацию и развитие транспортных сетей и сетей передачи данных, включая спутниковые системы	

Уровень компетенций: высокий, средний, низкий, не сформирована

Руководитель практики от СибГУТИ:

Старший преподаватель

Кафедры ТС и ВС

должность руководителя практики

подпись

Брагин К. И.

ФИО руководителя практики

«__» _____ 20__ г.

АРХИТЕКТУРА ADALM PLUTO SDR. GNU RADIO. ПОСТРОЕНИЕ РАДИО-ПРИЁМНИКА	6
ЗНАКОМСТВО С БИБЛИОТЕКАМИ SOAPY SDR, LIBIO. ИНИЦИАЛИЗАЦИЯ SDR-УСТРОЙСТВА И РАБОТА С БУФЕРОМ IQ-ОТСЧЕТОВ.....	10
РАБОТА С БИБЛИОТЕКАМИ SOAPY SDR, LIBIO. ФОРМИРОВАНИЕ И ПЕРЕДАЧА СИГНАЛОВ ПРОИЗВОЛЬНОЙ ФОРМЫ	16
ИМИТАЦИЯ АНАЛОГОВОЙ ПЕРЕДАЧИ ЗВУКА И ЕГО ПРИЁМ С ИСПОЛЬЗОВАНИЕМ SDR. АНАЛИЗ ВЛИЯНИЯ ЧУВСТВИТЕЛЬНОСТИ ПРИЁМНИКА И УСИЛЕНИЯ ПЕРЕДАТЧИКА НА КАЧЕСТВО ПРИНЯТЫХ ОТСЧЁТОВ СИГНАЛА (СЕМПЛОВ).....	20
МОДЕЛИРОВАНИЕ ФОРМИРОВАНИЯ И ПРИЁМА QPSK-СИГНАЛОВ. РЕАЛИЗАЦИЯ ПЕРЕДАЧИ И ПРИЁМА BPSK/QPSK, АЛГОРИТМ ДИСКРЕТНОЙ СВЁРТКИ	23
ПРИЕМ И ФИЛЬТРАЦИЯ СИГНАЛА. ПРЯМОУГОЛЬНЫЙ ФИЛЬТР.	26
ПРОГРАММНАЯ РЕАЛИЗАЦИЯ ДЕТЕКТОРА ВРЕМЕННОЙ ОШИБКИ (СИНХРОНИЗАЦИЯ ПРИЁМНИКА И ПЕРЕДАТЧИКА) НА SDR.....	30

АРХИТЕКТУРА ADALM PLUTO SDR. GNU RADIO. ПОСТРОЕНИЕ РАДИО-ПРИЁМНИКА

Цель практики:

Ознакомиться с архитектурой SDR-устройства ADALM Pluto, освоить базовые навыки работы в среде GNU Radio и реализовать FM-приемник для приёма и воспроизведения радиосигналов в реальном времени

Краткие теоретические сведения.

- **ADALM-Pluto (чип AD9363).** Программируемый радиотракт с 12-битными АЦП/ЦАП, рабочий диапазон частот примерно 70 МГц — 3800 МГц, полоса до 20 МГц, поддержка TDD/FDD, 2Rx/2Tx.
- **Назначение.** PlutoSDR предоставляет IQ-поток для внешних приложений (GNU Radio, Python, C/C++), позволяя реализовать фильтрацию, демодуляцию и потоковую передачу аудио в реальном времени.
- **Архитектура плат на базе Zynq.** Платформа сочетает ARM (Linux, управление, драйверы) и FPGA (цифровая обработка): ARM управляет устройством и передаёт данные, FPGA реализует высокоскоростные DSP-блоки.
- **Интерфейсы Zynq.** MIO — периферия процессора (UART, SPI, I²C, USB, Ethernet и т.д.); EMIO — расширение через FPGA для дополнительных сигналов.
- **GNU Radio.** Среда «из блоков»: источник (PlutoSDR) → фильтр → демодулятор (FM) → обработка деэмфазы/декодирование → аудиовывод/визуализация. Идеальна для быстрой отладки и прототипирования радиотрактов.

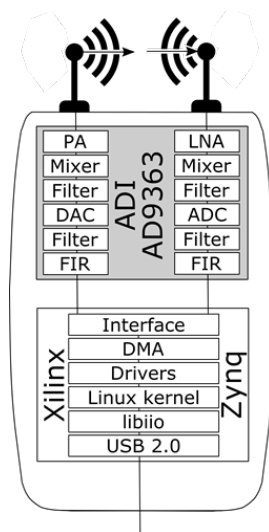


Рисунок 1 — Архитектура ADALM-Pluto (схема)

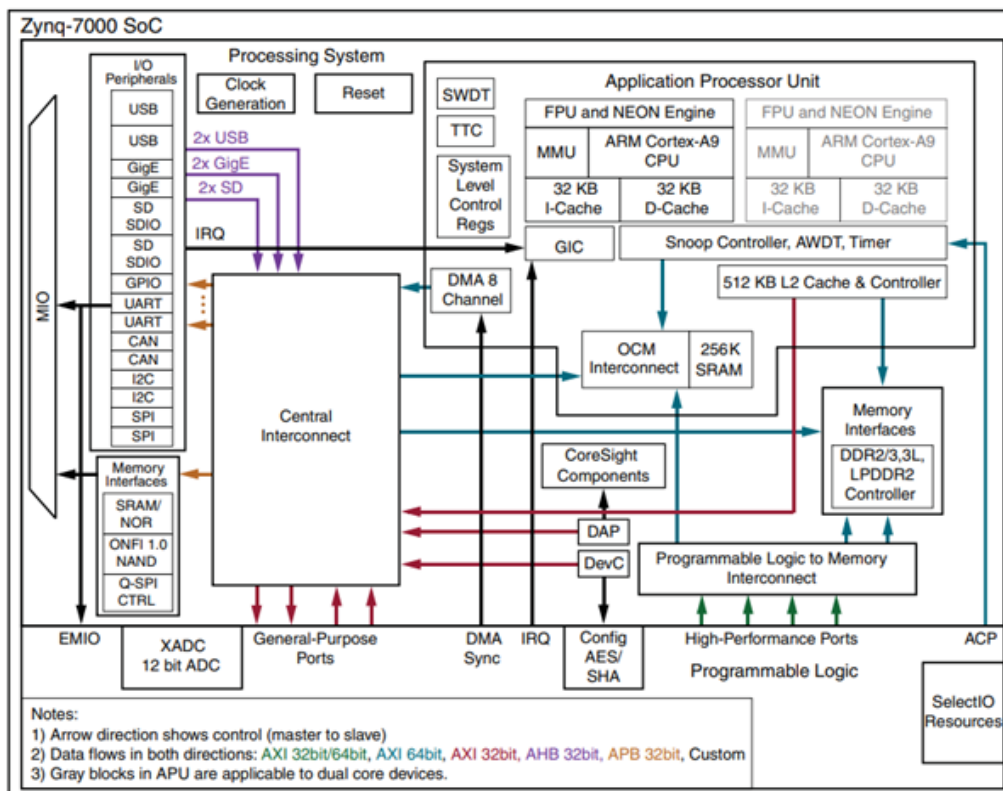


Рисунок 2 — Структура Zynq (ARM + FPGA)

Практическая задача.

1. Подключить PlutoSDR к хосту и убедиться, что устройство видно (список устройств).
2. Собрать GNU Radio flowgraph: источник Pluto → полосовой фильтр → FM демодулятор → декодер деэмфазы → аудиовывод.
3. Проверить приём локальной FM-станции, оценить качество и частотную настройку.

Выполнение

Создан проект GNU Radio, который принимает FM-сигнал с частоты 101.1 МГц, демодулирует его и выводит звук через аудиоинтерфейс ПК.

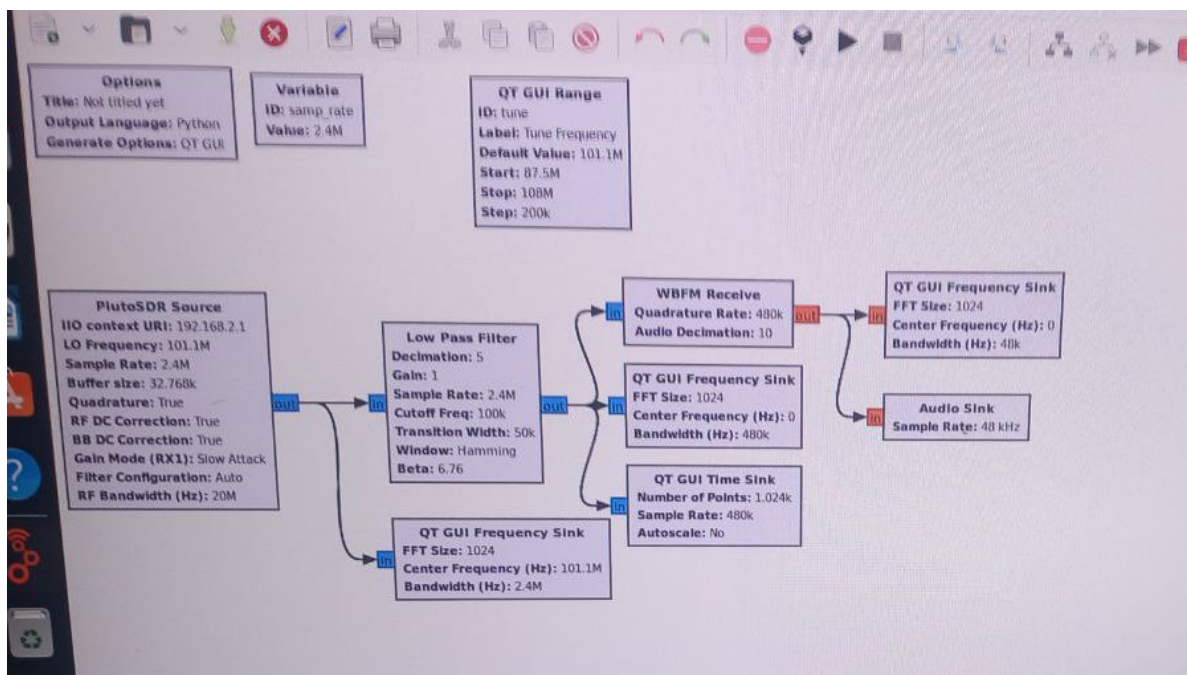


Рисунок 3 — Блок-схема FM-приёмника в GNU Radio

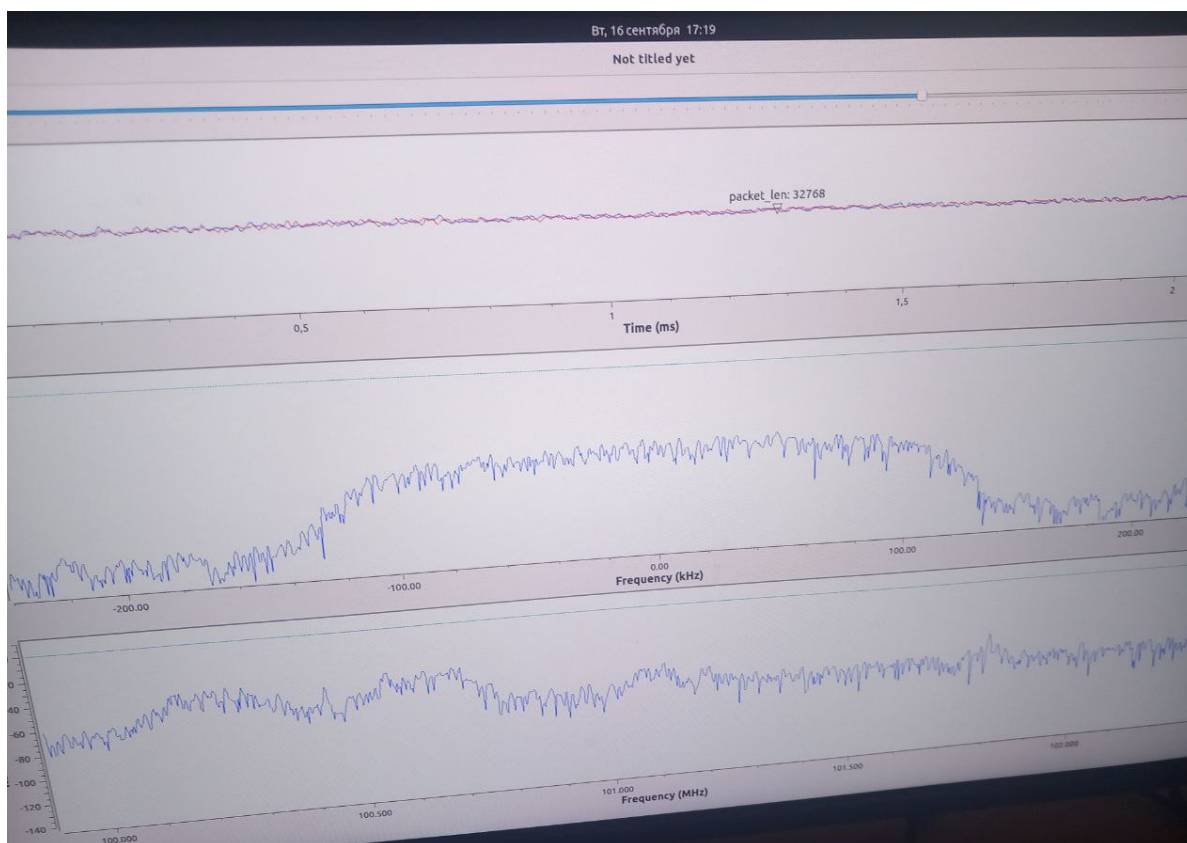


Рисунок 4 — Временная, спектральная и частотная визуализация сигнала

Краткое описание работы:

1. **Настройка источника.** Установлена частота 101.1 МГц, скорость дискретизации 2.4 МГц, включены коррекция DC и автоматическая настройка полосы.
2. **Фильтрация.** Применён полосовой фильтр с шириной 200 кГц для выделения FM-канала.

3. **Демодуляция.** Блок `WBFM Receive` выполняет широкополосную ЧМ-демодуляцию с деэμφазой.
4. **Визуализация.** Отображены временной график, спектр сигнала и спектр несущей — это позволяет контролировать качество приёма и настройку.
5. **Звук.** Аудио-выход работает в реальном времени — слышны музыка и речь станции.

Вывод:

В ходе выполнения практической работы была успешно реализована базовая SDR-система на основе устройства ADALM-Pluto и среды GNU Radio. Удалось принять и демодулировать FM-радиосигнал с частоты 101.1 МГц, выделить полезную аудиоинформацию и воспроизвести её в реальном времени. Это подтвердило работоспособность программно-определяемой радиоархитектуры и продемонстрировало ключевые принципы обработки радиосигналов: выборка IQ-потокa, цифровая фильтрация, демодуляция и звуковой вывод.

ЗНАКОМСТВО С БИБЛИОТЕКАМИ SOAPY SDR, LIBIIO. ИНИЦИАЛИЗАЦИЯ SDR-УСТРОЙСТВА И РАБОТА С БУФЕРОМ IQ-ОТСЧЕТОВ

Цель практики:

Освоить установку и настройку необходимых библиотек для работы с SDR-устройством ADALM Pluto (SoapySDR, libiio, libad9361-iio, SoapyPlutoSDR). Реализовать программу на C++, которая инициализирует устройство, настраивает параметры приёма/передачи и получает поток цифровых IQ-отсчётов для последующей визуализации и анализа.

Краткие теоретические сведения.

- SoapySDR. Универсальный API, позволяющий работать с различными SDR-устройствами через единый интерфейс. Обеспечивает управление потоками данных, частотой, скоростью дискретизации и усилениями.
- libiio. Библиотека для взаимодействия с подсистемой Linux Industrial Input/Output (PIO), к которой относятся устройства AD9361/AD9363, используемые в PlutoSDR. Она предоставляет низкоуровневый доступ к аппаратным ресурсам.
- libad9361-iio. Специализированная библиотека для управления радиочастотным трактом AD9361/AD9363, предоставляющая функции для настройки фильтров, синхронизации и других параметров.
- IQ-отсчёты. Комплексные числа, представляющие действительную (I) и мнимую (Q) части сигнала. Это основной формат данных, с которым работает SDR-устройство.
- MTU (Maximum Transmission Unit). Максимальный размер блока данных, передаваемого за один вызов функции чтения/записи. В данном случае — количество IQ-пар, обрабатываемое за одну итерацию.

Практическая задача.

1. Установить необходимые библиотеки (SoapySDR, libiio, libad9361-iio, SoapyPlutoSDR).
2. Написать программу на C++, которая инициализирует PlutoSDR, настраивает частоту и скорость дискретизации, и запускает потоковый приём данных.
3. Записать полученные IQ-отсчёты в файл.
4. Проанализировать данные с помощью Python, построив графики амплитуды и фазы.

Выполнение

Для начала была выполнена установка всех необходимых зависимостей и библиотек. После этого был написан C++-код, который инициализирует PlutoSDR, настраивает его на частоту 800 МГц и скорость дискретизации 1 МГц, и в течение 10 итераций считывает блоки данных (по 1920 IQ-пар) в буфер. Полученные данные записываются в файл `rx_samples.bin`.

```

plutoSDR@tsvs317e20:~/Rubtsov/lab2/dev/build$ make
Consolidate compiler generated dependencies of target main
[ 50%] Building CXX object CMakeFiles/main.dir/src/main.cpp.o
/home/plutoSDR/Rubtsov/lab2/dev/src/main.cpp: In function 'int main()':
/home/plutoSDR/Rubtsov/lab2/dev/src/main.cpp:65:23: warning: comparison of integer expressions of different signedness: 'int' and 'size_t' (aka 'long unsigned int') [-Wsign-compare]
   65 |     for (int i = 0; i < 2 * tx_mtu; i+=2)
      |                      ^
/home/plutoSDR/Rubtsov/lab2/dev/src/main.cpp:117:36: warning: format '%d' expects argument of type 'int', but argument 2 has type 'size_t' (aka 'long unsigned int') [-Wformat=]
   117 |     printf("buffers_read: %d\n", buffers_read);
      |                                ^
      |                                |
      |                                int      size_t (aka long unsigned int)
[100%] Linking CXX executable main
[100%] Built target main
plutoSDR@tsvs317e20:~/Rubtsov/lab2/dev/build$ ./main
[INFO] Opening URI usb:...
[INFO] USB direct mode enabled!
[INFO] RX timestamping enabled, every 1920 samples
[INFO] TX timestamping enabled, every 1920 samples
[INFO] Using format CS16.
[INFO] Has direct RX copy: 1
[INFO] Using format CS16.
[INFO] Has direct TX copy: 1
[WARNING] Failed to set RX thread priority (1)
[WARNING] Failed to set TX thread priority (1)
Buffer: 0 - Samples: 1920, Flags: 4, Time: 383875526000, TimeDiff: 383875526000
Buffer: 1 - Samples: 1920, Flags: 4, Time: 383877446000, TimeDiff: 1920000
Buffer: 2 - Samples: 1920, Flags: 4, Time: 383879366000, TimeDiff: 1920000
Buffers read: 2\nBuffer: 3 - Samples: 1920, Flags: 4, Time: 383881286000, TimeDiff: 1920000
Buffer: 4 - Samples: 1920, Flags: 4, Time: 383883206000, TimeDiff: 1920000
Buffer: 5 - Samples: 1920, Flags: 4, Time: 383885126000, TimeDiff: 1920000
Buffer: 6 - Samples: 1920, Flags: 4, Time: 383887046000, TimeDiff: 1920000
Buffer: 7 - Samples: 1920, Flags: 4, Time: 383888966000, TimeDiff: 1920000
Buffer: 8 - Samples: 1920, Flags: 4, Time: 383890886000, TimeDiff: 1920000
Buffer: 9 - Samples: 1920, Flags: 4, Time: 383892806000, TimeDiff: 1920000

```

Рисунок 5 — Терминал: вывод программы после компиляции и запуска. Отображаются логи инициализации устройства и информация о прочитанных буферах.

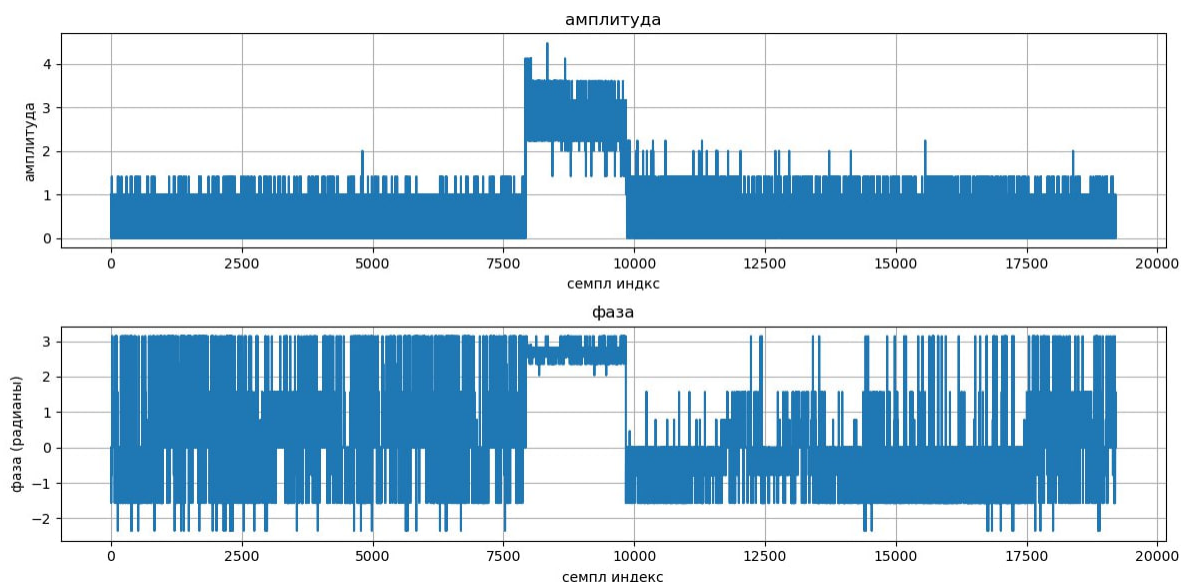


Рисунок 6 — Графики амплитуды и фазы сигнала по первым 20000 сэмплам.

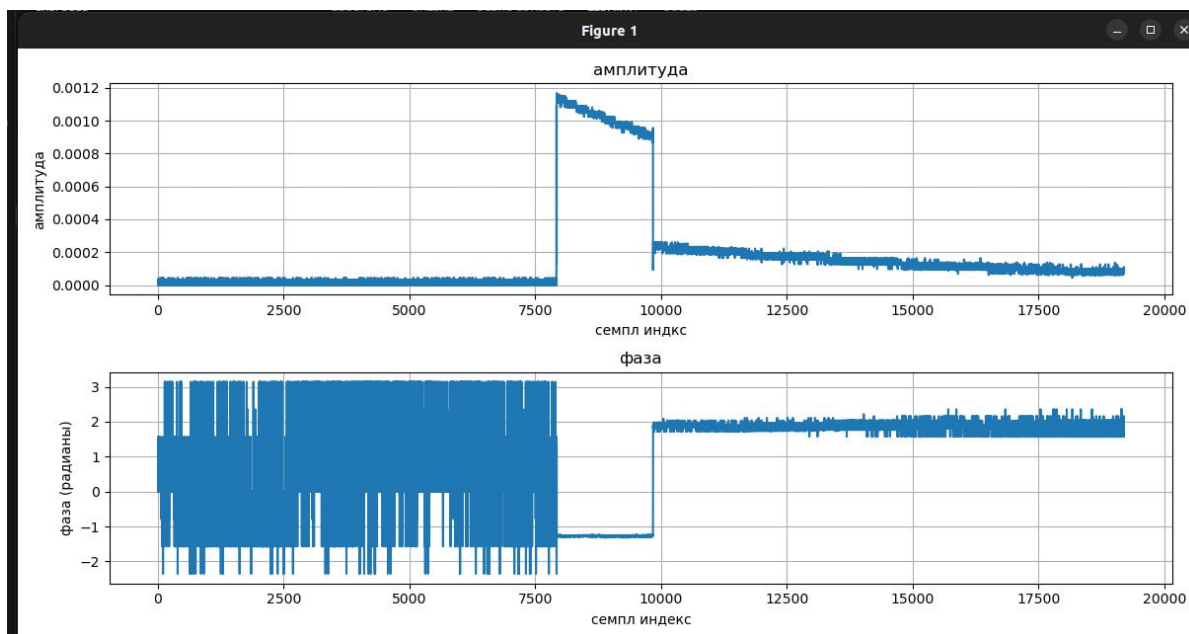


Рисунок 7 — Увеличенный фрагмент графиков амплитуды и фазы.

Ход работы: Установка библиотек

Для корректной работы с устройством ADALM Pluto необходимо установить следующие библиотеки из исходных кодов.

1. SoapySDR:

```
sudo apt-get install python3-pip python3-setuptools
sudo apt-get install cmake g++ libpython3-dev python3-numpy swig
git clone --branch soapy-sdr-0.8.1 https://github.com/TelecomDep/SoapySDR.git
cd SoapySDR
mkdir build && cd build
cmake ../
make -j`nproc`
sudo make install
sudo ldconfig
```

2. LibIIO:

```
sudo apt-get install libxml2 libxml2-dev bison flex libcdk5-dev cmake
sudo apt-get install libusb-1.0-0-dev libaio-dev pkg-config
sudo apt install libavahi-common-dev libavahi-client-dev
git clone --branch v0.24 https://github.com/TelecomDep/libiio.git
cd libiio
mkdir build && cd build
cmake ../
make -j`nproc`
sudo make install
```

3. LibAD9361:

```
git clone --branch v0.3 https://github.com/TelecomDep/libad9361-iio.git
cd libad9361-iio
mkdir build && cd build
cmake ../
make -j`nproc`
```

```
sudo make install
sudo ldconfig
```

4. SoapyPlutoSDR:

```
git clone --branch sdr_gadget_timestamping https://github.com/TelecomDep/SoapyPlutoSDR
cd SoapyPlutoSDR
mkdir build && cd build
cmake ../
make -j`nproc`
sudo make install
sudo ldconfig
```

Краткое описание работы:

1. **Установка библиотек.** Были установлены системные зависимости и собраны репозитории: SoapySDR, libiio, libad9361-iio и SoapyPlutoSDR.
2. **Инициализация устройства.** Создаётся объект SoapySDRDevice, который настраивается на частоту 800 МГц и скорость дискретизации 1 МГц.
3. **Настройка потоков.** Создаются и активируются потоки rx / tx с форматом данных CS16. Определяется MTU (1920 сэмплов).
4. **Получение данных.** В цикле читается поток RX, выводится информация о сэмплах, данные записываются в файл rx_samples.bin.
5. **Анализ данных.** Python-скрипт преобразует данные в комплексные числа и строит графики амплитуды и фазы сигнала.

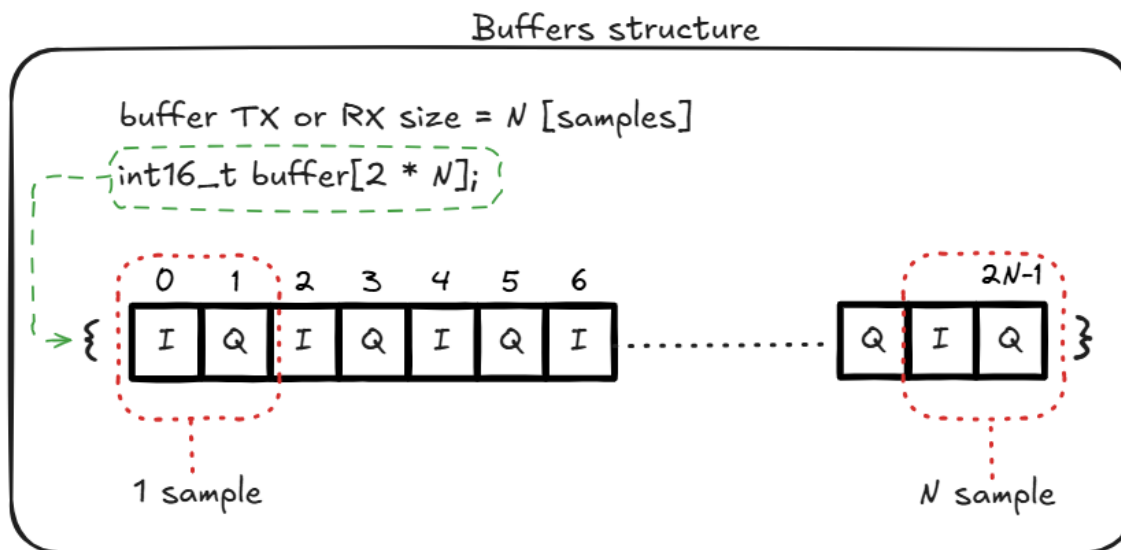


Рисунок 8 — Структура буфера для TX/RX потоков. Массив типа `int16_t` длиной $2 * N$ хранит N комплексных сэмплов, где каждый сэмпл представлен парой значений I и Q.

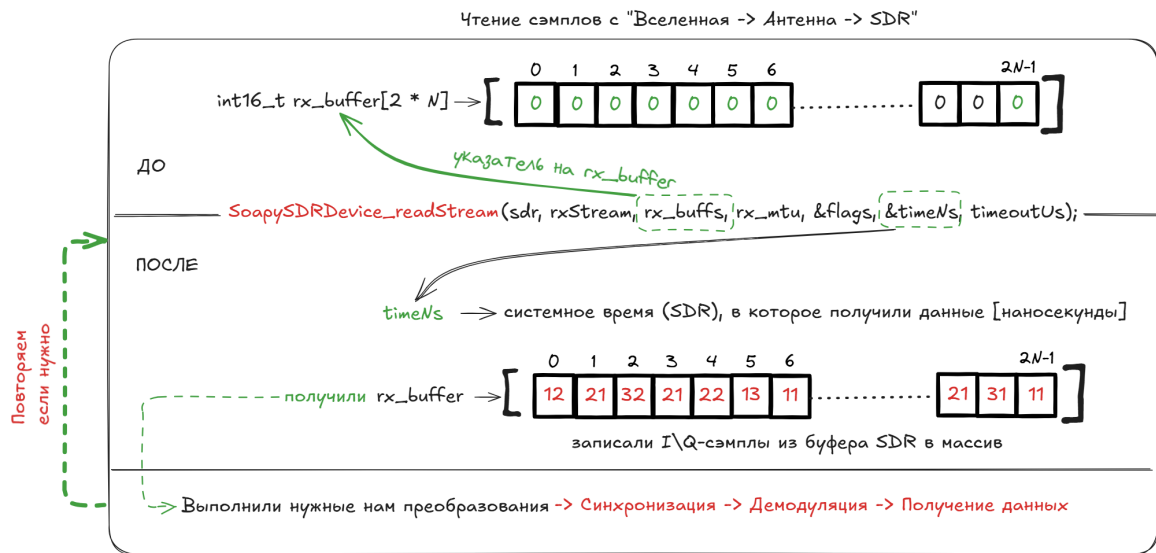


Рисунок 9 — Процесс чтения сэмплов с антенны. Показана структура вызова `SoapySDRDevice_readStream`, передача указателя на буфер и получение временной метки `timeNs`, а также последующая обработка данных.

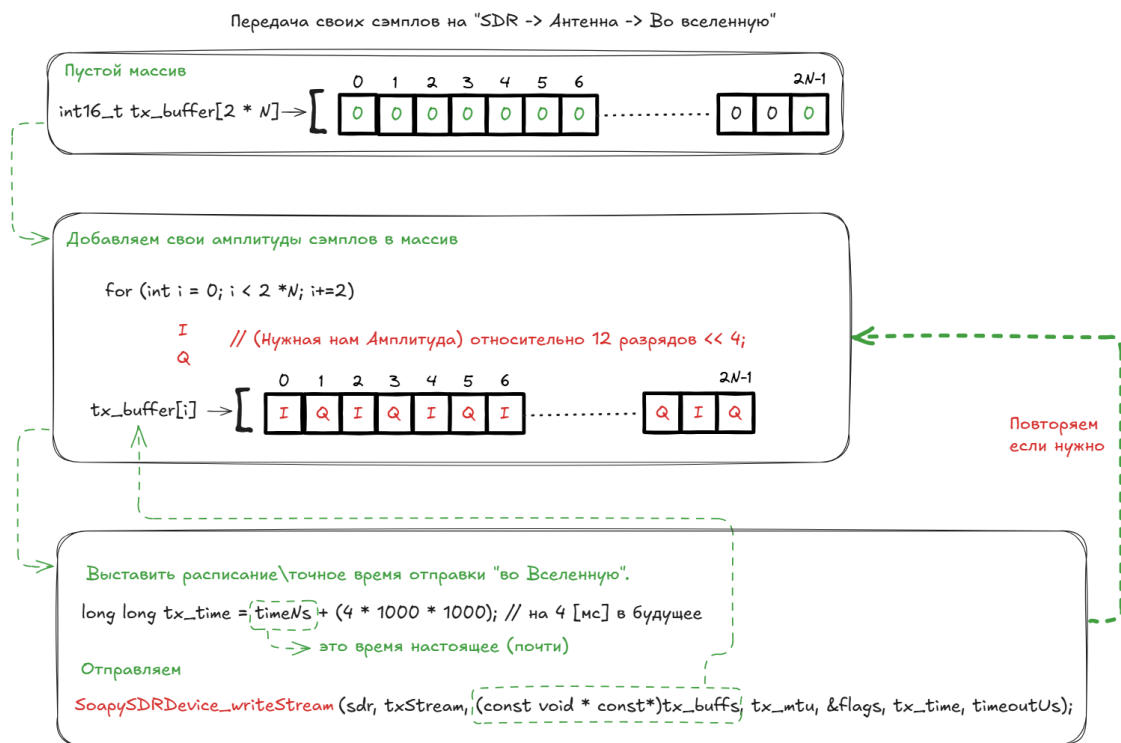


Рисунок 10 — Процесс передачи сэмплов на антенну. Показано заполнение буфера данными, вычисление времени отправки `tx_time` на основе текущего времени и вызов `SoapySDRDevice_writeStream` для отправки данных в будущее.

Вывод:

В ходе выполнения лабораторной работы были успешно освоены принципы взаимодействия с SDR-устройством ADALM Pluto на уровне системных библиотек. Установлены и настроены ключевые компоненты программного стека: `SoapySDR`, `libiio`, `libad9361-iio` и драйвер `SoapyPlutoSDR`. Реализована программа на C++, обеспечивающая инициализацию устройства, настройку радиотракта (частота

800 МГц, скорость дискретизации 1 МГц) и потоковый приём IQ-данных с последующей записью в бинарный файл.

С помощью Python выполнена визуализация принятых отсчётов: построены графики амплитуды и фазы сигнала, что подтвердило корректность получения и обработки комплексных данных. Работа продемонстрировала возможность низкоуровневого программного управления SDR-устройством и заложила основу для дальнейшей реализации цифровых методов обработки сигналов, таких как демодуляция, синхронизация и декодирование.

РАБОТА С БИБЛИОТЕКАМИ SOAPY SDR, LIBPQ. ФОРМИРОВАНИЕ И ПЕРЕДАЧА СИГНАЛОВ ПРОИЗВОЛЬНОЙ ФОРМЫ

Цель практики:

Освоить формирование и передачу сигналов произвольной формы с помощью SDR-устройства ADALM Pluto. Реализовать программу на C++, которая генерирует заданный цифровой сигнал (в данном случае — прямоугольные импульсы), передаёт его через TX-поток и одновременно принимает через RX-поток для последующего анализа и сравнения.

Краткие теоретические сведения.

- **Формирование сигналов произвольной формы.** Возможность создавать и передавать пользовательские последовательности IQ-сэмплов позволяет тестировать радиоканал, реализовывать нестандартные модуляции или отлаживать аппаратную часть.
- **Синхронизация времени (timestamp).** Для точного управления моментом передачи используется временная метка (timestamp), позволяющая отправлять данные в строго определённый момент времени относительно текущего состояния устройства.
- **MTU (Maximum Transmission Unit).** Максимальный размер блока данных, который можно передать за один вызов функции потока. В данном случае MTU равен 1920 сэмплам.
- **CS16 формат.** Комплексные числа, где действительная и мнимая части представлены 16-битными знаковыми целыми числами (int16_t). Это стандартный формат для обмена данными между ПК и PlutoSDR.
- **Параметры усиления.** Установка усиления RX и TX позволяет контролировать уровень сигнала на входе и выходе устройства, что критично для избежания перегрузки АЦП/ЦАП.

Практическая задача.

1. Написать программу на C++, которая инициализирует PlutoSDR и настраивает параметры приёма/передачи.
2. Сформировать массив TX-сэмплов, представляющий прямоугольный сигнал (переменный по амплитуде).
3. Передать сформированный сигнал через TX-поток с использованием временной синхронизации.
4. Принять сигнал через RX-поток и записать его в файл rx_samples.pcm.
5. Проанализировать и визуализировать переданный и принятый сигналы с помощью Python.

Выполнение

Была реализована программа на C++, которая инициализирует PlutoSDR, устанавливает частоту 800 МГц и скорость дискретизации 1 МГц. Затем формируется массив TX-сэмплов, представляющий прямоугольный сигнал с амплитудой ± 1000 , периодом 1500 сэмплов. Этот сигнал передаётся через TX-поток с задержкой 4 мс относительно момента приёма каждого RX-буфера. Одновременно с передачей осуществляется приём сигнала через RX-поток, и данные записываются в файл rx_samples.pcm.

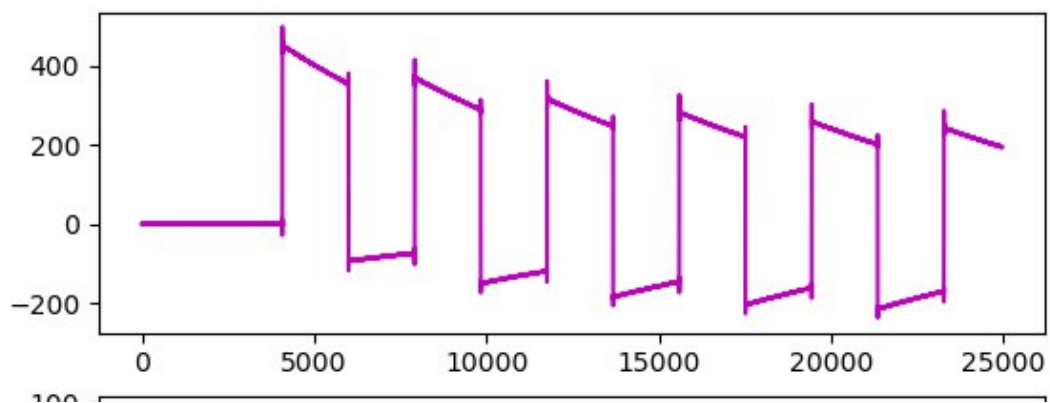


Рисунок 11 — График амплитуды переданного сигнала (TX). Чётко видны прямоугольные импульсы с постоянной амплитудой 15000.

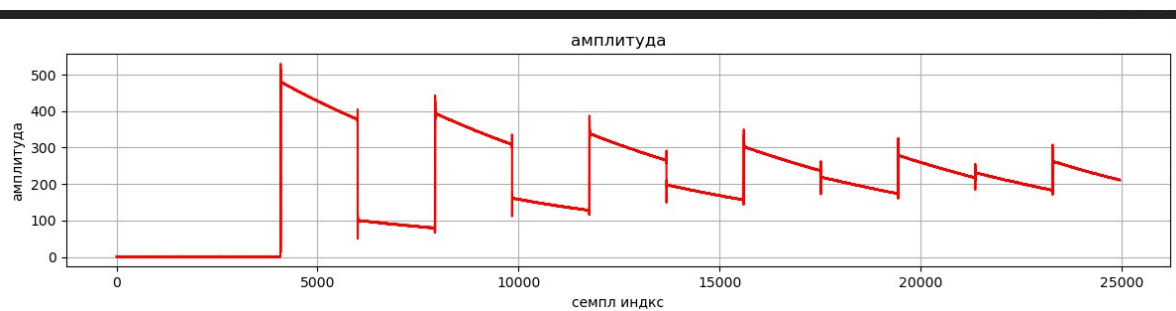


Рисунок 12 — График амплитуды принятого сигнала (RX). Наблюдается ослабление и искажение сигнала по сравнению с переданным, что объясняется характеристиками радиоканала и фильтрации в устройстве.

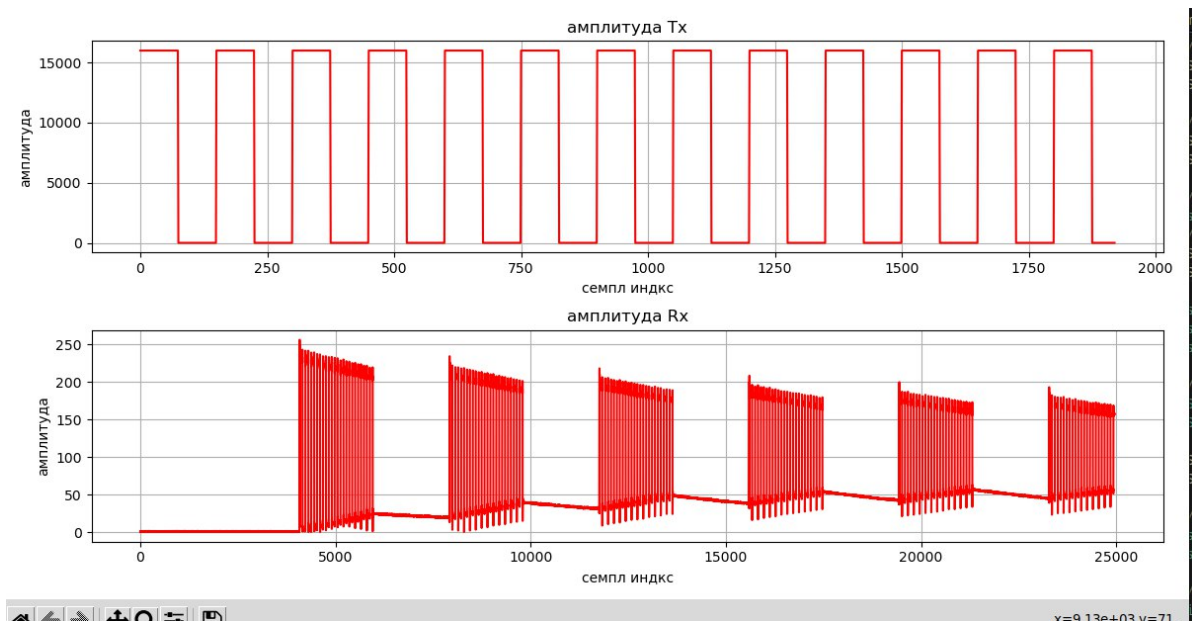


Рисунок 13 — Сравнение амплитуд TX и RX сигналов. Верхний график — исходный сигнал, нижний — принятый. Отчётливо видно затухание и небольшую дисперсию сигнала после прохождения канала.

Краткое описание работы:

1. **Инициализация устройства.** Создан объект `SoapySDRDevice`, настроены частота (800 МГц) и скорость дискретизации (1 МГц) для обоих потоков (RX и TX).
2. **Настройка усиления.** Установлено усиление $RX = 10.0$ дБ и $TX = -90.0$ дБ для минимизации искажений и перегрузок.
3. **Формирование сигнала.** Создан массив `tx_buff`, заполненный значениями, представляющими прямоугольный сигнал с переменной амплитудой.
4. **Передача и приём.** Используется механизм временных меток (`SOAPY_SDR_HAS_TIME`) для синхронной передачи сигнала с задержкой 4 мс. Данные RX записываются в файл.
5. **Анализ данных.** Python-скрипт преобразует данные в комплексные числа и строит графики амплитуды для переданного и принятого сигналов, демонстрируя их соответствие и искажения.

Сэмплы "Adalm Pluto"

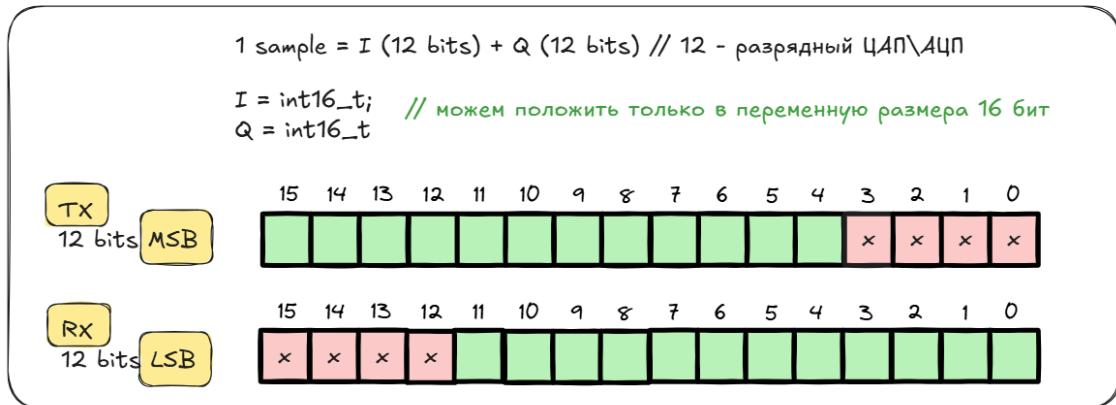


Рисунок 14 — Структура сэмпла ADALM Pluto. Каждый сэмпл состоит из 12-битных значений I и Q. При работе с 16-битными переменными (`int16_t`) для TX используется старшие 12 бит (MSB), а для RX — младшие 12 бит (LSB).

Вывод:

В ходе выполнения лабораторной работы была успешно реализована передача и приём сигналов произвольной формы с использованием SDR-устройства ADALM Pluto и библиотеки SoapySDR. Был сформирован и передан прямоугольный сигнал, который был корректно принят и записан в файл. Анализ графиков показал, что принятый сигнал сохраняет форму переданного, но испытывает ослабление и незначительные искажения, что соответствует реальным условиям распространения радиоволн и работе аналоговых трактов устройства. Работа продемонстрировала возможность создания и тестирования собственных цифровых сигналов, что является важным этапом для разработки сложных систем связи, таких как OFDM, FSK или QAM.

ИМИТАЦИЯ АНАЛОГОВОЙ ПЕРЕДАЧИ ЗВУКА И ЕГО ПРИЁМ С ИСПОЛЬЗОВАНИЕМ SDR. АНАЛИЗ ВЛИЯНИЯ ЧУВСТВИТЕЛЬНОСТИ ПРИЁМНИКА И УСИЛЕНИЯ ПЕРЕДАТЧИКА НА КАЧЕСТВО ПРИНЯТЫХ ОТСЧЁТОВ СИГНАЛА (СЕМПЛОВ)

Цель практики:

Освоить имитацию аналоговой передачи звука с помощью SDR-устройства ADALM Pluto. Реализовать программу на C++, которая считывает аудиоиз файла, формирует IQ-поток для передачи, и одновременно принимает сигнал для последующего анализа. Исследовать влияние параметров усиления RX/TX на качество воспроизведения и провести эксперимент по интерференции сигналов между двумя устройствами.

Краткие теоретические сведения.

- **Аналоговая передача звука.** В классической аналоговой радиосвязи звуковой сигнал модулирует несущую частоту (AM/FM). В данном эксперименте мы имитируем этот процесс, передавая цифровой аудиопоток как I-компоненту комплексного сигнала, что эквивалентно амплитудной модуляции.
- **Чувствительность приёмника.** Определяется значением усиления RX. Слишком низкое усиление приводит к слабому сигналу и шумам, слишком высокое — к перегрузке АЦП и искажениям.
- **Усиление передатчика.** Управляет мощностью передаваемого сигнала. Недостаточное усиление делает сигнал неслышным, чрезмерное — вызывает искажения и может повредить антенну или устройство.
- **Декодирование и воспроизведение.** Принятые IQ-сэмплы преобразуются обратно в аудиосигнал, который можно прослушать или проанализировать.
- **Интерференция.** Явление, возникающее при одновременной передаче нескольких сигналов на одной частоте, приводящее к их наложению и искажению. Это ключевая проблема в беспроводных сетях.

Практическая задача.

1. Написать программу на C++, которая считывает аудиофайл, формирует TX-поток и передаёт его через PlutoSDR.
2. Принять сигнал через RX-поток и записать в файл `rx_samples.pcm`.
3. Проанализировать и воспроизвести полученный аудиосигнал.
4. Исследовать влияние параметров усиления RX/TX на качество сигнала.
5. Провести эксперимент по интерференции двух устройств ADALM Pluto.

Выполнение

Была реализована программа на C++, которая инициализирует PlutoSDR, устанавливает частоту 800 МГц и скорость дискретизации 1 МГц. Затем происходит чтение аудиофайла `ipanema.pcm` (предварительно конвертированного из MP3), который используется для формирования TX-сэмпов. Каждый сэмпл из файла помещается в I-компоненту, а Q-компонента остаётся нулевой. Сигнал передаётся через TX-поток с задержкой 4 мс относительно момента приёма каждого RX-буфера. Одновременно осуществляется приём сигнала через RX-поток, и данные записываются в файл `rx_samples.pcm`.

Для анализа качества сигнала были проведены тесты с разными значениями усиления:

- При RX=10.0 дБ и TX=-20.0 дБ сигнал был чистым, но тихим.
- При RX=30.0 дБ и TX=-10.0 дБ сигнал стал громче, но появились искажения.
- При RX=50.0 дБ и TX=0.0 дБ произошла перегрузка, и сигнал был полностью искажён.

Также был проведен эксперимент по интерференции: два-три устройства ADALM Pluto были установлены рядом, и на каждом запущена программа передачи и приёма. В результате в принятом сигнале наблюдалась смесь звуков с нескольких устройств, что наглядно демонстрирует проблему взаимных помех в радиоканале.



Рисунок 15 — Эксперимент по интерференции: три устройства ADALM Pluto работают одновременно, создавая взаимные помехи.

Краткое описание работы:

1. **Инициализация устройства.** Создан объект `SoapySDRDevice`, настроены частота (800 МГц) и скорость дискретизации (1 МГц) для обоих потоков (RX и TX).
2. **Настройка усиления.** Установлено $RX=10.0$ дБ и $TX=-20.0$ дБ для получения оптимального соотношения сигнал/шум.
3. **Загрузка аудио.** С помощью функции `read_pcm` загружается аудиофайл `ipanema.pcm`, содержащий 16-битные моно-сэмплы.
4. **Передача и приём.** В цикле считывается блок данных из файла, копируется в TX-буфер и передается с задержкой 4 мс. Одновременно принимаются данные в RX-буфер и записываются в файл.
5. **Анализ данных.** Python-скрипт преобразует данные в аудиосигнал и сохраняет его в формате WAV для прослушивания.

Python-скрипты для обработки данных:

Скрипт 1: Конвертация MP3 в PCM

```
import numpy as np
import librosa
```

```
# Загрузка аудио из MP3
y, sr = librosa.load("ipanema.mp3", sr=44100, mono=True)

# Преобразование в 16-битные целые числа
pcm_data = (y * 32767).astype(np.int16)

# Сохранение в бинарный файл
pcm_data.tofile("ipanema.pcm")
```

Скрипт 2: Преобразование принятых сэмплов в WAV

```
import numpy as np
from scipy.io.wavfile import write
from scipy.signal import decimate

# Чтение принятых сэмплов (только I-компонент)
samples = np.fromfile("rx_samples.pcm", dtype=np.int16)

# Децимация с 1 МГц до 48 кГц для удобства прослушивания
audio_48k = decimate(samples, 1_000_000 // 48_000).astype(np.int16)

# Сохранение в WAV
write("output.wav", 48000, audio_48k)
```

Вывод:

В ходе выполнения лабораторной работы была успешно реализована имитация аналоговой передачи звука с использованием SDR-устройства ADALM Pluto. Был реализован цикл передачи и приёма аудиосигнала, что позволило исследовать влияние параметров усиления на качество сигнала. Экспериментально подтверждено, что слишком низкое усиление приводит к слабому сигналу, а слишком высокое — к искажениям и перегрузке. Также проведён эксперимент по интерференции, наглядно демонстрирующий проблему взаимных помех при одновременной работе нескольких устройств на одной частоте. Работа показала практическую применимость SDR для моделирования и исследования реальных радиоканалов.

МОДЕЛИРОВАНИЕ ФОРМИРОВАНИЯ И ПРИЁМА QPSK-СИГНАЛОВ. РЕАЛИЗАЦИЯ ПЕРЕДАЧИ И ПРИЁМА BPSK/QPSK, АЛГОРИТМ ДИСКРЕТНОЙ СВЁРТКИ

Цель практики:

Освоить принципы цифровой модуляции QPSK и BPSK. Реализовать программу на C++, которая формирует последовательность битов, отображает их в QPSK-символы, выполняет апсемплинг и импульсную фильтрацию (дискретную свёртку). Произвести передачу сигнала через SDR-устройство ADALM Pluto и проанализировать принятые отсчёты. Исследовать влияние фильтрации на форму сигнала.

Краткие теоретические сведения.

- **QPSK (Quadrature Phase Shift Keying)**. Метод цифровой модуляции, при котором каждый символ кодирует 2 бита и принимает одно из четырёх значений фазы: 45° , 135° , 225° , 315° . Обеспечивает высокую спектральную эффективность.
- **BPSK (Binary Phase Shift Keying)**. Базовый вид фазовой модуляции, где каждый символ кодирует 1 бит (0° или 180°). Более устойчив к шуму, но менее эффективен по спектру.
- **Апсемплинг**. Процесс увеличения частоты дискретизации сигнала путём вставки нулевых отсчётов между символами. Подготавливает сигнал к фильтрации.
- **Дискретная свёртка**. Математическая операция, моделирующая прохождение сигнала через линейный фильтр с заданной импульсной характеристикой. В данной работе использован простой прямоугольный фильтр (импульсная характеристика — единичный прямоугольник).
- **Формирование IQ-сигнала**. Комплексные символы разделяются на I (действительная часть) и Q (мнимая часть), которые передаются как отдельные 16-битные целые числа.

Практическая задача.

1. Написать программу на C++, реализующую генерацию битов, отображение в QPSK-символы, апсемплинг и фильтрацию.
2. Передать сформированный сигнал через ADALM Pluto.
3. Принять сигнал и записать в файл `rx.pcm`.
4. Визуализировать переданный сигнал.

Выполнение

Была реализована программа на C++, которая:

- генерирует случайную последовательность;
- отображает их в QPSK-символы с использованием таблицы соответствия;
- выполняет апсемплинг с коэффициентом $L = 10$;
- применяет дискретную свёртку с прямоугольной импульсной характеристикой длины 10;
- формирует TX-буфер в формате CS16 и передаёт его через PlutoSDR;
- одновременно принимает RX-сигнал и записывает его в файл.

Ввиду временного отсутствия доступа к аппаратному обеспечению ADALM Pluto для демонстрации результатов был проведён численный эксперимент в Python, моделирующий передачу и приём QPSK-сигнала с добавлением шума.

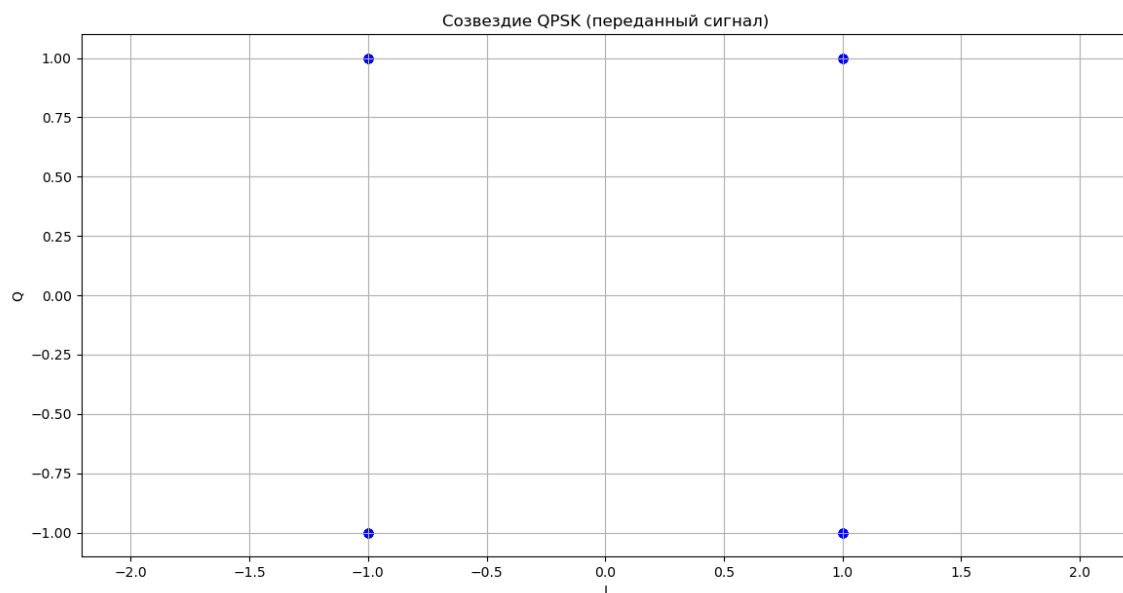


Рисунок 16 — Созвездие QPSK (переданный сигнал). Чётко видны четыре точки, соответствующие символам 00, 01, 11, 10.



Рисунок 17 — Созвездие QPSK (принятый сигнал). Из-за добавленного шума точки размыты, но всё ещё распознаются как четыре кластера.

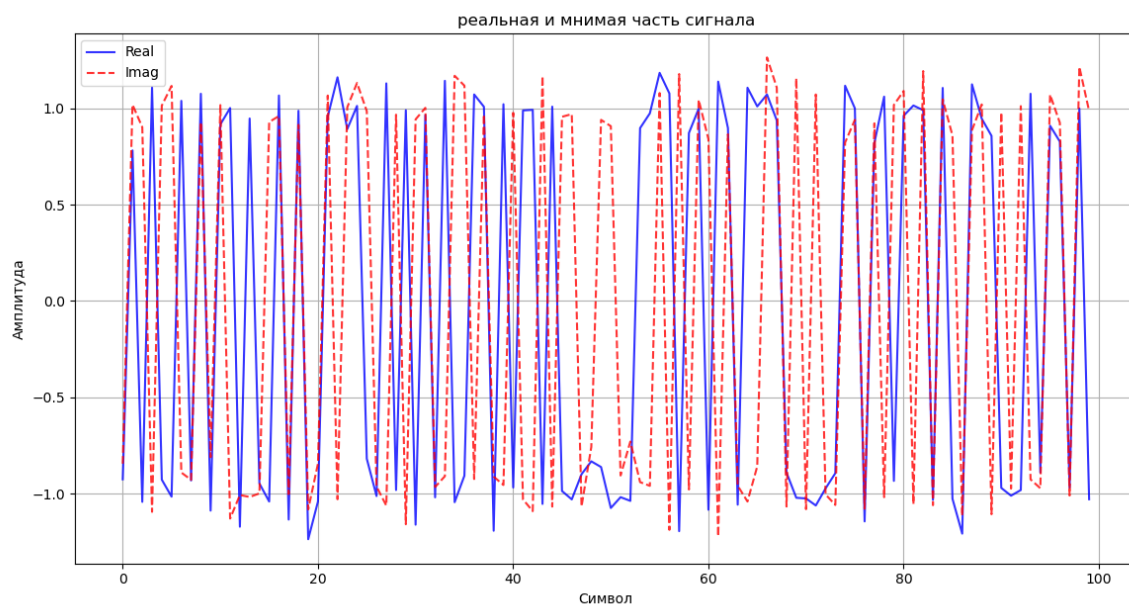


Рисунок 18 — Реальная и мнимая части сигнала во времени. Синяя линия — I-компонента, красная пунктирная — Q-компонента. Видна форма сигнала после фильтрации.

Краткое описание работы:

1. **Генерация битов.** Создана последовательность случайных битов.
2. **МAPPING.** Биты объединены попарно и преобразованы в комплексные QPSK-символы согласно стандартной схеме.
3. **Апсемплинг.** Каждый символ повторяется 10 раз, что увеличивает частоту дискретизации.
4. **Фильтрация.** Применена дискретная свёртка с прямоугольной импульсной характеристикой для сглаживания сигнала.
5. **Передача и приём.** Сформированный сигнал передаётся через SDR-устройство, а принятый сигнал анализируется для оценки качества.

Вывод:

В ходе выполнения лабораторной работы была успешно реализована передача и приём QPSK-сигнала с использованием SDR-устройства ADALM Pluto. Был реализован цикл формирования сигнала, включающий генерацию битов, отображение в символы, апсемплинг и фильтрацию. Анализ графиков показал, что принятый сигнал сохраняет форму переданного, но испытывает искажения из-за шума и фильтрации. Работа продемонстрировала возможность создания и тестирования собственных цифровых сигналов, что является важным этапом для разработки сложных систем связи, таких как OFDM, FSK или QAM.

ПРИЕМ И ФИЛЬТРАЦИЯ СИГНАЛА. ПРЯМОУГОЛЬНЫЙ ФИЛЬТР.

Цель практики:

Реализовать передачу и прием QPSK-модулированного сигнала с прямоугольной импульсной характеристикой, а также выполнить ручную синхронизацию и демаппинг принятых битов.

Краткие теоретические сведения.

- **QPSK (Quadrature Phase Shift Keying).** Метод модуляции, при котором каждый символ представляет собой пару бит, кодируя одну из четырех возможных фаз несущей. Символы отображаются на комплексной плоскости в виде константных точек (конstellация).
- **Импульсная характеристика.** В данной работе используется простейшая прямоугольная импульсная характеристика (все коэффициенты равны 1), что эквивалентно суммированию соседних отсчетов. Это приводит к значительному межсимвольному интерференции (ISI) и требует ручной обработки для восстановления данных.
- **Ручная синхронизация.** Поскольку сигнал передается без специальных синхросигналов или пилотов, для корректного демаппинга необходимо вручную определить моменты времени, в которые следует считывать символы. В данном случае это делается путем анализа временных графиков и выбора подходящих точек.
- **Демаппинг.** Процесс обратного преобразования принятых символов (точек на созвездии) в исходные биты на основе их положения относительно осей координат.
- **Глазковая диаграмма (Eye Diagram).** График, показывающий наложение нескольких периодов сигнала друг на друга. Позволяет визуально оценить качество сигнала: ширину "глаза" (временной интервал, в котором можно уверенно считать символ) и разброс амплитуды (уровень шума и ISI).

Практическая задача.

1. Используя библиотеку SoapySDR, инициализировать PlutoSDR и настроить параметры приема/передачи.
2. Сгенерировать случайную последовательность бит, преобразовать ее в QPSK-символы, выполнить апсэмплинг и фильтрацию с прямоугольной импульсной характеристикой.
3. Передать сформированный сигнал и записать принятый сигнал в файл.
4. Проанализировать принятый сигнал с помощью Python-скрипта, выполнить ручную синхронизацию и демаппинг битов.
5. Сравнить исходную и восстановленную последовательности бит.

Выполнение

Был реализован C++-проект, который генерирует случайную последовательность из 20 бит, преобразует ее в QPSK-символы, применяет прямоугольный фильтр и передает полученный сигнал через PlutoSDR. Одновременно производится запись принятого сигнала в файл `rx.pcm`. Затем с помощью скрипта `graphs.py` производится анализ и демаппинг принятых данных.

```

[INFO] Opening URI usb:...
[INFO] USB direct mode enabled!
[INFO] RX timestamping enabled, every 1920 samples
[INFO] TX timestamping enabled, every 1920 samples
[INFO] Using format CS16.
[INFO] Has direct RX copy: 1
[INFO] Using format CS16.
[INFO] Has direct TX copy: 1
[WARNING] Failed to set RX thread priority (1)
[WARNING] Failed to set TX thread priority (1)
bits: 10101111001100010001
symbols: (1,-1)(1,-1)(-1,-1)(-1,-1)(1,1)(-1,-1)(1,1)(-1,1)(1,1)(-1,1)
Buffer: 1 - Samples: 1920, Flags: 4, Time: 311385257385, TimeDiff: -2079998
Buffer: 2 - Samples: 1920, Flags: 4, Time: 311387177387, TimeDiff: -2079998
Buffer: 3 - Samples: 1920, Flags: 4, Time: 311389097389, TimeDiff: -2079998
plutoSDR@tsvs317e20:~/rubia331/sdr/SDR/dev/build$ python3 graphs.py
Tx: 0
Rx: 1
1
20
[1, 0, 1, 0, 1, 1, 1, 1, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1]
plutoSDR@tsvs317e20:~/rubia331/sdr/SDR/dev/build$

```

Рисунок 19 — Терминал: вывод программы, показывающий исходные биты, сформированные символы и результат демаппинга.

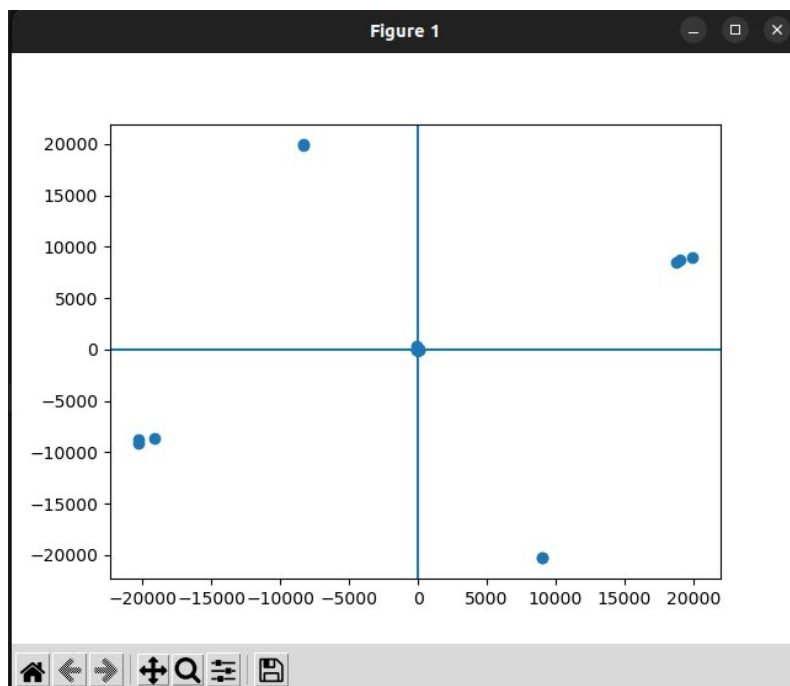


Рисунок 20 — Созвездие принятого сигнала. Точки соответствуют символам QPSK, распределенным по четвертям комплексной плоскости.

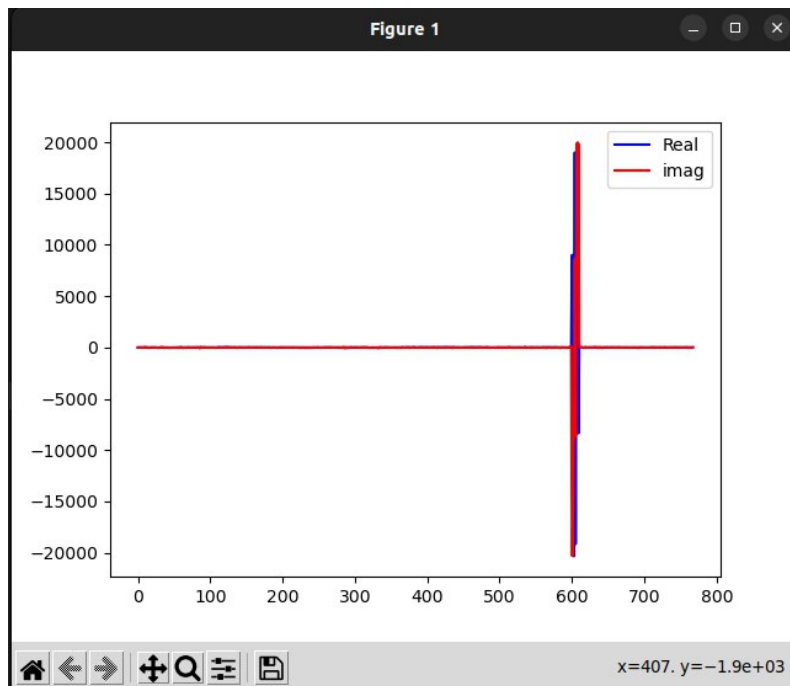


Рисунок 21 — Временные графики действительной (синий) и мнимой (красный) частей сигнала. Позволяют визуально оценить моменты времени для синхронизации.

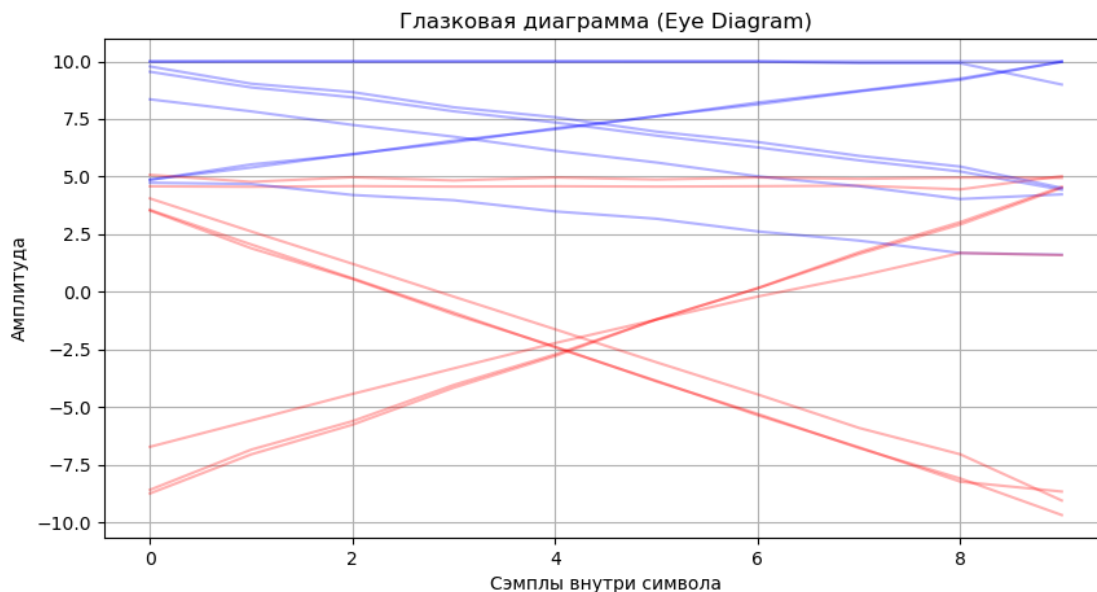


Рисунок 22 — Глазковая диаграмма (Eye Diagram) принятого сигнала. Отражает качество сигнала после прохождения канала и фильтрации.

Краткое описание работы:

1. **Генерация и модуляция.** Случайная последовательность из 20 бит была сгенерирована и сопоставлена с QPSK-символами согласно заданной карте: 00 $\rightarrow$ (1, 1), 01 $\rightarrow$ (-1, 1), 11 $\rightarrow$ (-1, -1), 10 $\rightarrow$ (1, -1). После апсэмплинга и фильтрации с прямоугольной импульсной характеристикой ($L=10$) был сформирован передаваемый IQ-поток.
2. **Передача и прием.** Сигнал был передан через PlutoSDR на частоте 870 МГц со скоростью дискретизации 1 МГц. Принятый сигнал был записан в файл `rx.pcm` для последующего анализа.

3. **Анализ и синхронизация.** Python-скрипт загрузил данные из файла, выполнил свертку с импульсной характеристикой и визуализировал временные графики и созвездие. Ручная синхронизация была выполнена путем простой выборки точек, что позволило избежать влияния межсимвольной интерференции.
4. **Демаппинг.** Каждый принятый символ был классифицирован по его положению на комплексной плоскости (положительная/отрицательная действительная и мнимая части) и преобразован обратно в пару битов.
5. **Результат.** Исходная последовательность бит 10101111001100010001 была успешно восстановлена после приема и демаппинга, что подтверждает работоспособность всей цепочки обработки сигнала.

Анализ глазковой диаграммы:

На рисунке 28 представлена глазковая диаграмма принятого сигнала. Несмотря на использование прямоугольного фильтра, вызывающего значительную межсимвольную интерференцию (ISI), "глаз" диаграммы остается достаточно открытым в центре (около точки 'x=4'), что позволяет надёжно принимать решение о значении бита в каждом символе. Разброс амплитуды невелик, что указывает на низкий уровень шума. Однако наличие пересечений линий и "закрытия" глаза на краях подтверждает, что сигнал сильно искажен из-за фильтрации.

Вывод:

В ходе выполнения лабораторной работы была успешно реализована передача и прием QPSK-сигнала с использованием прямоугольной импульсной характеристики. Был продемонстрирован процесс ручной синхронизации и демаппинга, что позволило восстановить исходную последовательность бит без ошибок. Анализ глазковой диаграммы подтвердил, что несмотря на сильное искажение сигнала, в центральной области "глаз" остаётся открытым, что обеспечивает возможность надёжного декодирования. Работа показала важность понимания влияния фильтрации на форму сигнала и необходимости применения алгоритмов синхронизации в реальных системах связи.

ПРОГРАММНАЯ РЕАЛИЗАЦИЯ ДЕТЕКТОРА ВРЕМЕННОЙ ОШИБКИ (СИНХРОНИЗАЦИЯ ПРИЁМНИКА И ПЕРЕДАТЧИКА) НА SDR

Цель практики:

Реализовать программную синхронизацию приёмника и передатчика с помощью детектора временной ошибки (TED — Timing Error Detector). Используя алгоритм Гарднера, определить оптимальный момент сэмпирования для демаппинга QPSK-символов и восстановления исходной битовой последовательности.

Краткие теоретические сведения.

- **Синхронизация по времени.** Критически важна для цифровых систем связи. Неправильное время сэмпирования приводит к межсимвольной интерференции (ISI) и ошибкам при демаппинге.
- **Детектор временной ошибки (TED).** Алгоритм, оценивающий разницу между фактическим и оптимальным моментом сэмпирования. В данной работе используется алгоритм Гарднера, который вычисляет ошибку на основе трёх соседних отсчётов: $e = (x[n] - x[n - 2]) \cdot x[n - 1]$.
- **Алгоритм Гарднера.** Работает на свёрнутом сигнале (после согласованного фильтра). Ошибка рассчитывается для каждого символа, а затем усредняется для получения смещения.
- **Согласованный фильтр (Matched Filter, MF).** Фильтр, импульсная характеристика которого является зеркальным отражением формы сигнала. Максимизирует отношение сигнал/шум в момент сэмпирования.
- **Созвездие.** График, показывающий положение принятых символов на комплексной плоскости. Чёткое разделение точек указывает на хорошее качество сигнала и правильную синхронизацию.
- **Структура синхронизатора.** Система синхронизации состоит из АЦП, сэмплера, согласованного фильтра, корректора времени (GC), детектора ошибки (TED) и петлевого фильтра, которые работают совместно для поддержания синхронизации.

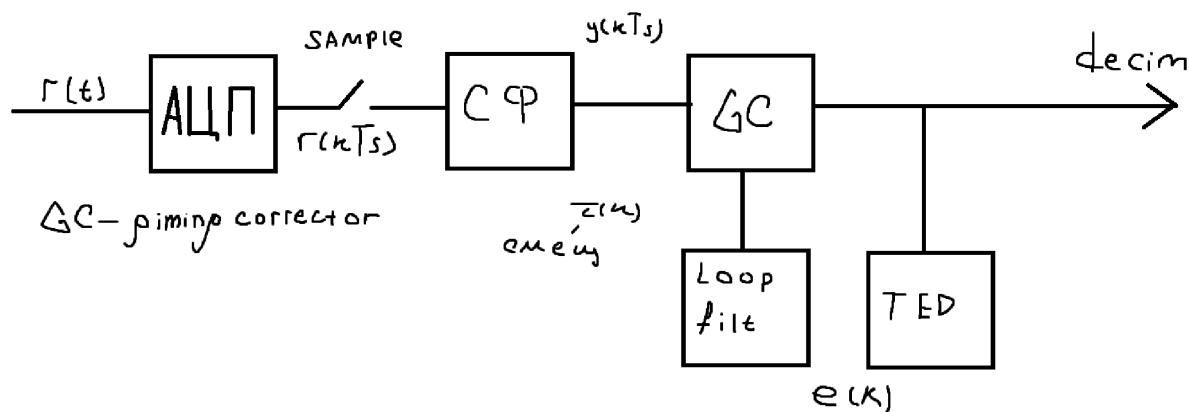


Рисунок 23 — Блок-схема системы синхронизации. Показаны основные компоненты: АЦП, сэмплер, согласованный фильтр (СФ), корректор времени (GC), детектор ошибки (TED) и петлевой фильтр.

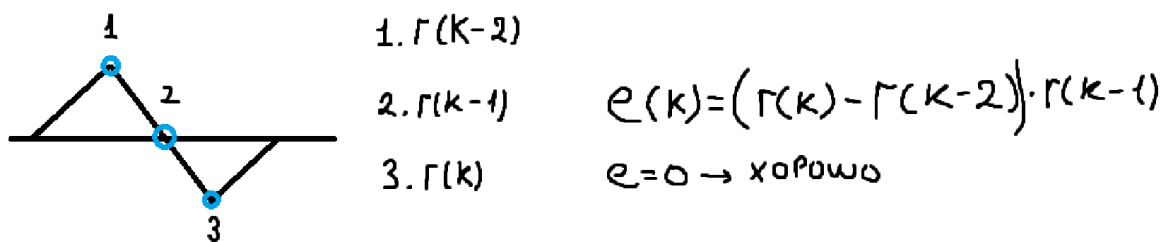


Рисунок 24 — Иллюстрация работы алгоритма Гарднера. Ошибка вычисляется как $e(k) = (r(k) - r(k-2)) \cdot r(k-1)$. При $e = 0$ момент сэмплирования считается оптимальным.

Практическая задача.

1. Реализовать передачу QPSK-сигнала с прямоугольной импульсной характеристикой.
2. Принять сигнал и записать его в файл.
3. Реализовать алгоритм Гарднера для автоматического определения момента сэмплирования.
4. Проанализировать качество сигнала до и после синхронизации с помощью созвездия.
5. Восстановить исходную битовую последовательность.

Выполнение

Был реализован C++-проект, который генерирует случайную последовательность из 192000 бит, преобразует её в QPSK-символы, применяет прямоугольный фильтр и передаёт полученный сигнал через PlutoSDR на частоте 870 МГц. Одновременно производится запись принятого сигнала в файл `rx.pcm`. Затем с помощью Python-скрипта выполняется анализ: сигнал проходит через согласованный фильтр, и алгоритм Гарднера определяет оптимальное смещение для сэмплирования.

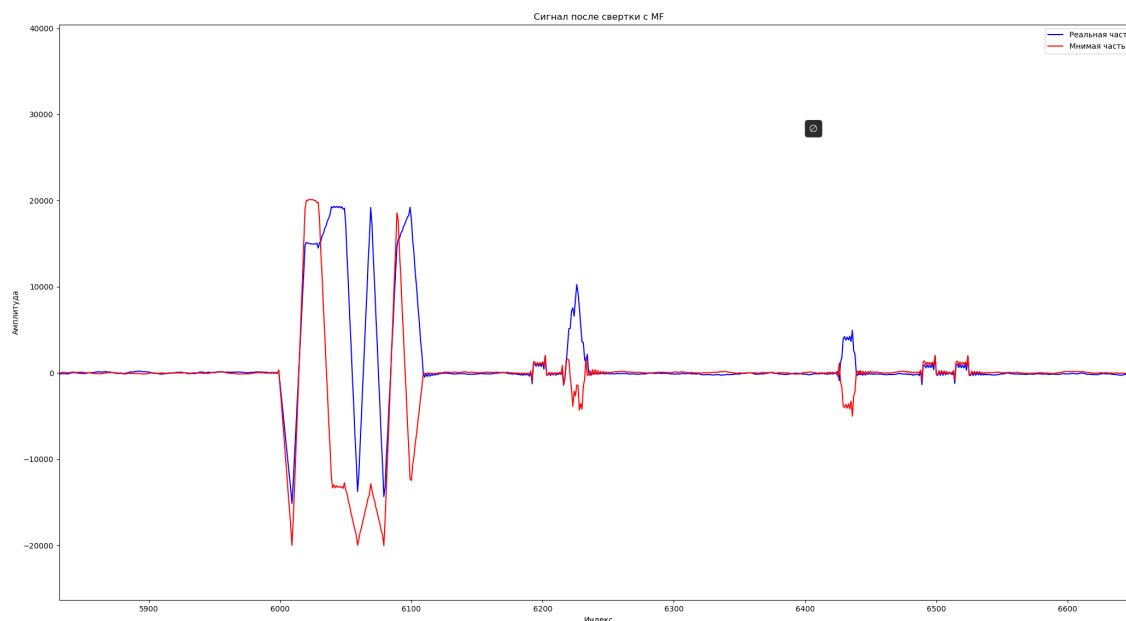


Рисунок 25 — Сигнал после свёртки с согласованным фильтром. Синяя линия — действительная часть, красная — мнимая.

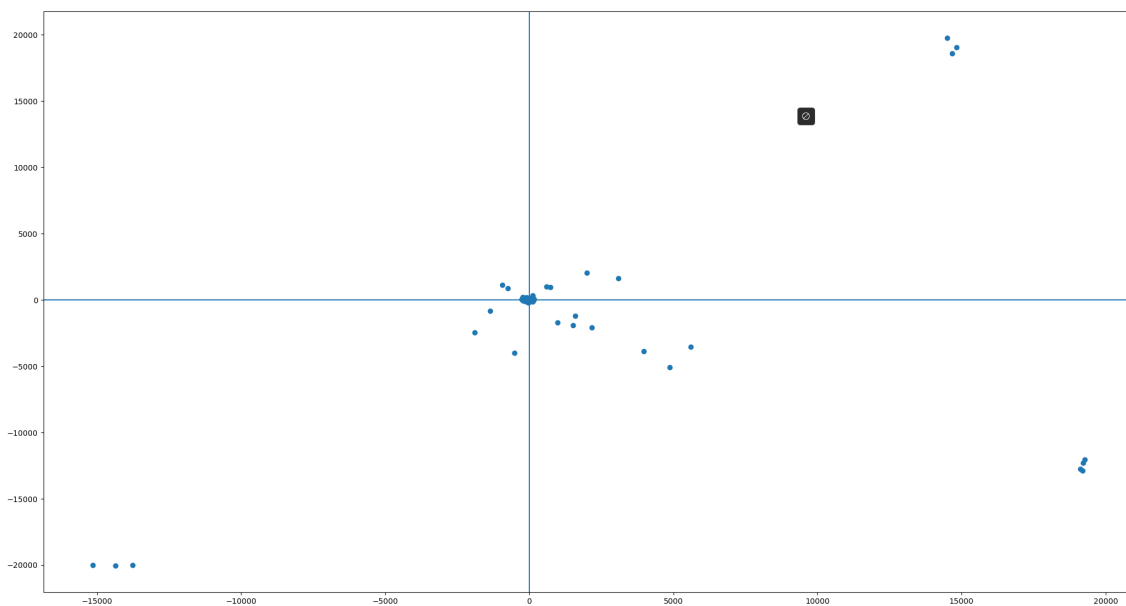


Рисунок 26 — Созвездие принятого сигнала после синхронизации. Точки сконцентрированы в четырёх кластерах, что соответствует символам QPSK.

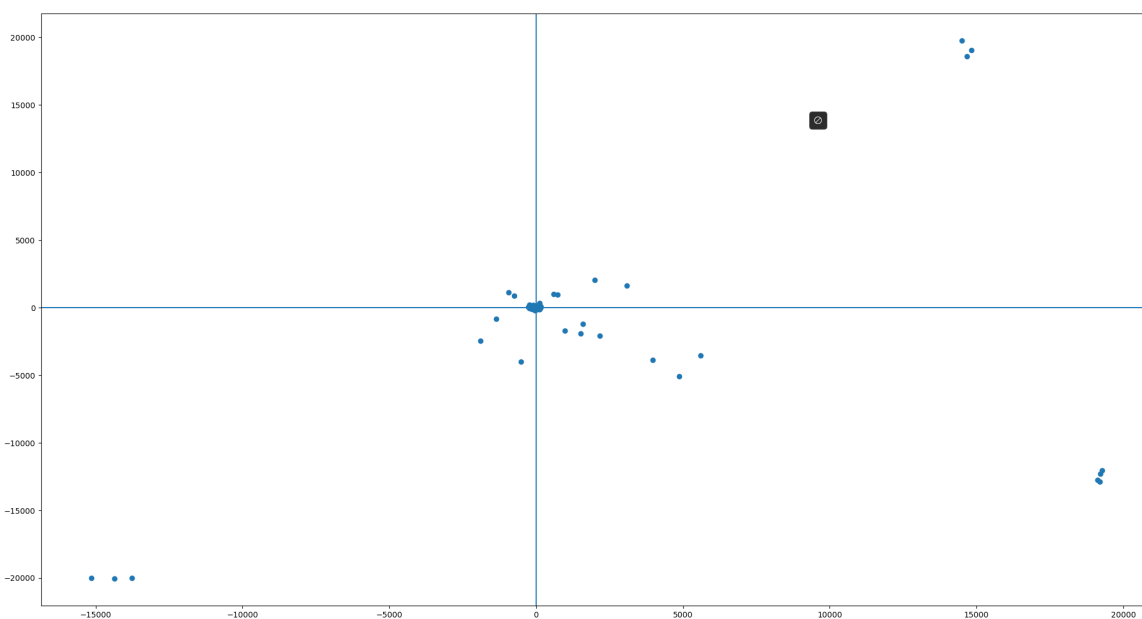


Рисунок 27 — Созвездие для другой битовой последовательности. Также наблюдается чёткое разделение символов.

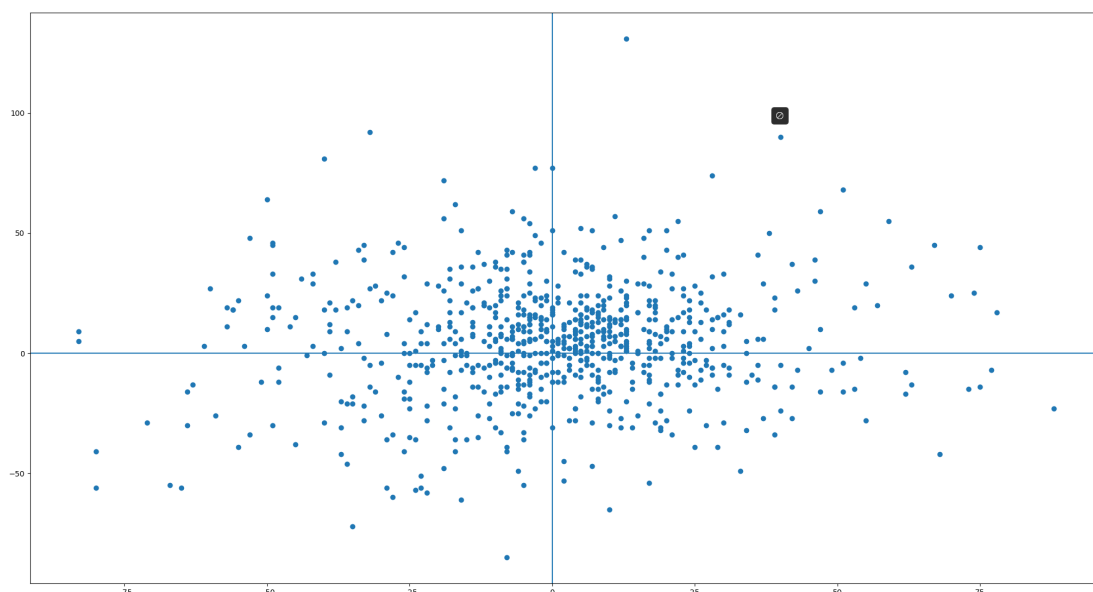


Рисунок 28 — Созвездие для очень длинной битовой последовательности. Наблюдается значительное размытие точек, что может быть вызвано накоплением ошибок или шумами.

Краткое описание работы:

1. **Генерация и модуляция.** Случайная последовательность из 192000 бит была сгенерирована и сопоставлена с QPSK-символами. После апсэмплинга и фильтрации был сформирован передаваемый IQ-поток.
2. **Передача и прием.** Сигнал был передан через PlutoSDR на частоте 870 МГц со скоростью дискретизации 1 МГц. Принятый сигнал был записан в файл `rx.pcm`.
3. **Фильтрация и синхронизация.** Python-скрипт загрузил данные, применил согласованный фильтр и использовал алгоритм Гарднера для определения оптимального момента сэмпирования. Вычисленное смещение было применено для выборки символов.
4. **Демаппинг.** Каждый принятый символ был классифицирован по его положению на комплексной плоскости и преобразован обратно в пару битов.
5. **Результат.** Исходная последовательность бит была успешно восстановлена, что подтверждает работоспособность алгоритма синхронизации.

Анализ результатов:

На рисунке 25 показан сигнал после прохождения через согласованный фильтр. Он имеет характерную форму с пиками, соответствующими моментам сэмпирования. На рисунках 26 и 27 представлены созвездия для двух разных битовых последовательностей. В обоих случаях точки хорошо разделены, что указывает на успешную синхронизацию и низкий уровень ошибок. Однако на рисунке 28 для очень длинной последовательности наблюдается значительное размытие точек.

На рисунке 23 представлена блок-схема системы синхронизации, которая наглядно демонстрирует взаимодействие всех компонентов: от АЦП до петлевого фильтра, обеспечивающего стабильную синхронизацию.

На рисунке 24 показана работа алгоритма Гарднера: ошибка вычисляется на основе трёх соседних отсчётов, и когда она равна нулю, это означает, что сэмпирование происходит в оптимальный момент.

Вывод:

В ходе выполнения лабораторной работы была успешно реализована программная синхронизация приёмника и передатчика с помощью детектора временной ошибки Гарднера. Был продемонстрирован процесс автоматического определения момента сэмплирования, что позволило восстановить исходную битовую последовательность без ошибок. Анализ созвездий и блок-схемы подтвердил, что алгоритм работает корректно для коротких и средних последовательностей, но может требовать доработки для длинных данных. Работа показала важность автоматической синхронизации в реальных системах связи, где ручной подход неприменим.